Anagrammes (récursivité)

CodEx — Exercices de programmation

Deux chaînes de même longueur sont des anagrammes si l'on peut écrire l'une en utilisant tous les caractères de l'autre, quitte à les déplacer : `chien` et `niche` sont des anagrammes, `louve` et `poule` ne le sont pas.

On utilise une approche récursive : si les deux chaînes sont de longueur 1, on renvoie `True` ou `False` selon qu'elles sont égales ou non ; sinon, on teste si le premier caractère de la première se trouve aussi dans la seconde : si oui, on recommence sur les deux chaînes privées de la première apparition de ce caractère ; si non, on renvoie `False`.

Écris deux fonctions (les chaînes sont non vides) :

— `supprime_premier(chaine, cible)` renvoie un couple `(present, chaine_sans_cible)` : `present` indique si le caractère `cible` est dans `chaine`, et `chaine_sans_cible` est la chaîne privée de la première apparition de `cible` (ou la même chaîne s'il n'y est pas) ;

— `anagrammes(chaine1, chaine2)` renvoie `True` si les deux chaînes sont des anagrammes, `False` sinon.

Exemples : `supprime_premier('ukulélé', 'u')` vaut `(True, 'kulélé')`, `supprime_premier('ukulélé', 'é')` vaut `(True, 'ukullé')`, `supprime_premier('ukulélé', 'a')` vaut `(False, 'ukulélé')` ; `anagrammes('chien', 'niche')` et `anagrammes('énergie noire', 'reine ignorée')` valent `True`, `anagrammes('louve', 'poule')` vaut `False`.